



Proyecto 1: Hoja de Cálculo Simple



Curso: Algoritmo y Estructura de Datos

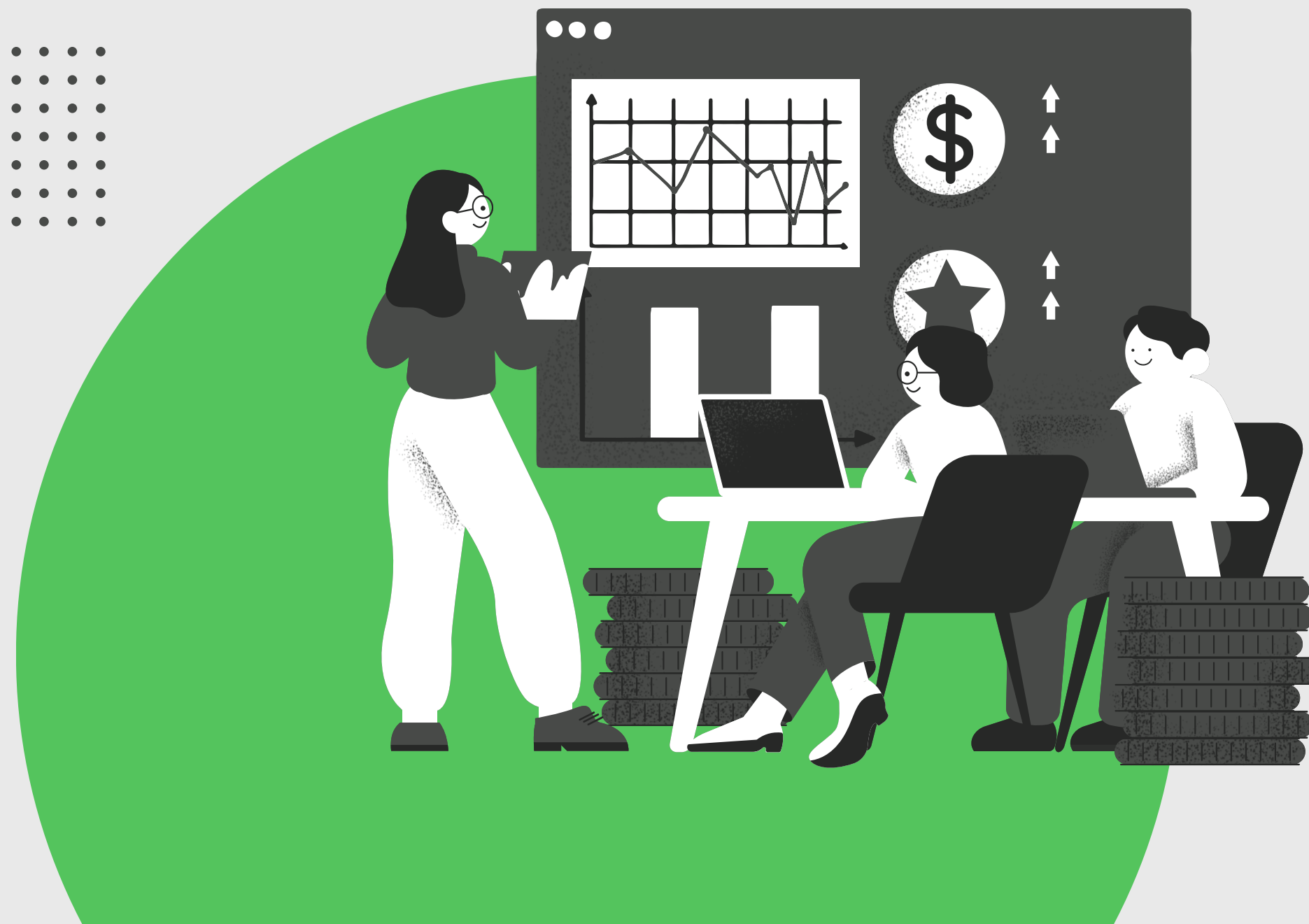
Profesor: Ojeda Rios, Brenner Humberto

Integrantes:

- Arana Espinoza, Fabian Nicolás
- Cartagena Miranda, Lucia Jimena
- Sanchez Cerna, Ary Fernando



Table of Contents



- ✦ Motivación y descripción del problema
- ✦ Diseño de la estructura de datos
- ✦ Recorrido del código
- ✦ Demostración en vivo de la aplicación

Motivación y descripción del problema

¿Por qué una matriz dispersa es más eficiente que una densa en este contexto?



La matriz dispersa solo consume memoria proporcional al número de celdas que realmente contienen datos. En aplicaciones donde el vacío es la norma (como el caso de excel que nos pide le proyecto), las matrices dispersas transforman un problema de memoria imposible en una solución ligera y escalable.

Uso de Memoria: Denso vs Disperso

Basado en una rejilla de 1,000 filas x 702 columnas con solo 1,000 celdas con datos.

Matriz Densa

~ 22.5 MB (Estático)

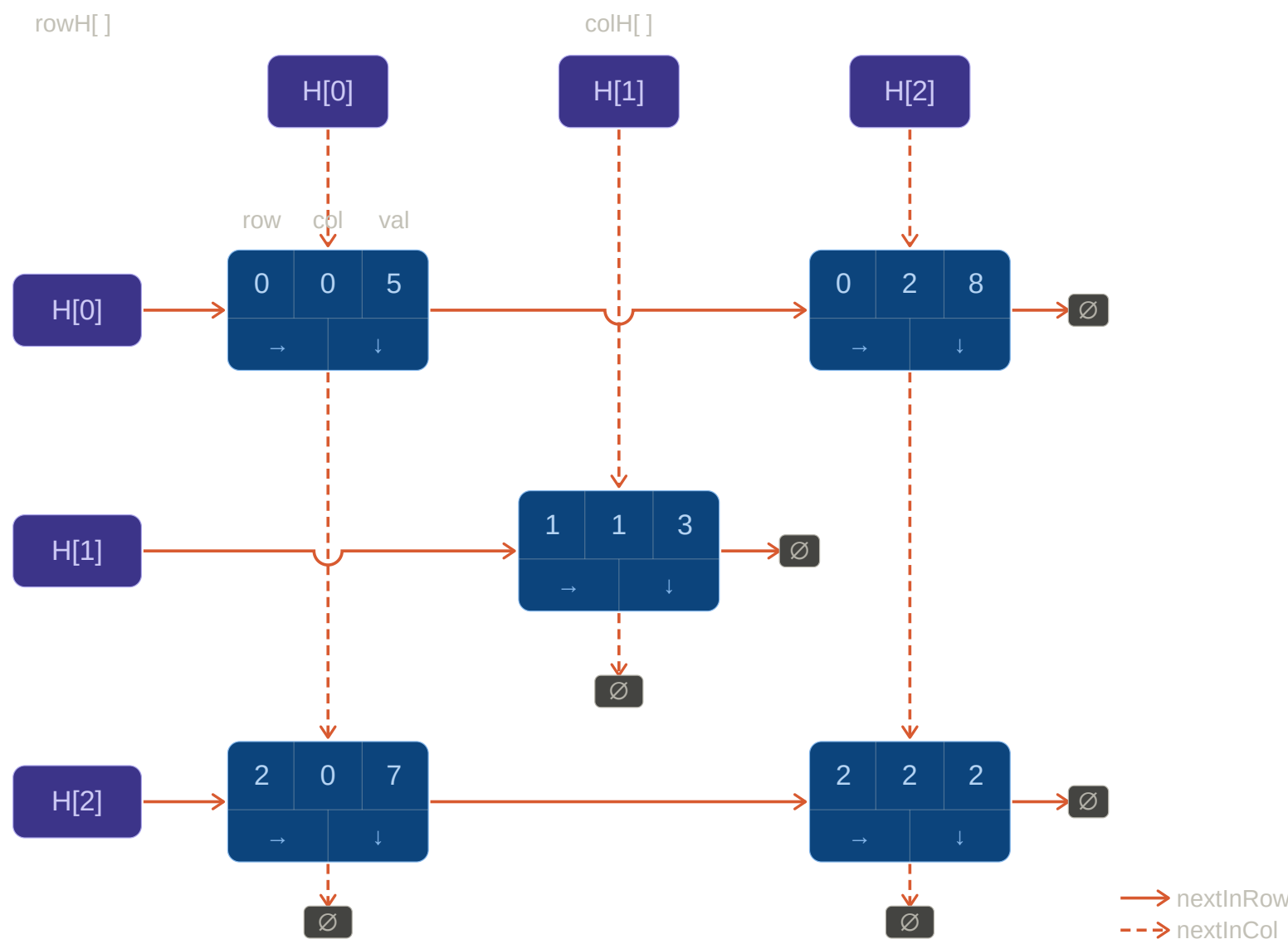
Matriz Dispersa

~ 54 KB (Dinámico)

La matriz densa requiere memoria para los 702,000 punteros de `std::string` incluso si están vacíos. La matriz dispersa solo consume memoria por cada `CellNode` creado.



Diseño de la estructura de datos



La estructura `CellNode` se compone de dos partes principales:

- Campos de datos:
 - **row (Fila)**
 - **col (Columna)**
 - **value (Valor)**
- Punteros de enlace:
 - **nextInRow**
 - **nextInCol**

Diseño de la estructura de datos

Complejidad Temporal: La complejidad depende de la cantidad de celdas ocupadas en las filas o columnas.

Definamos:

- **Kr:** Número de celdas ocupadas en una fila específica.
- **Kc:** Número de celdas ocupadas en una columna específica.

Funciones:

- **consultarCelda:**
 - **Complejidad:** $O(Kr)$
 - **Justificación:** La función extrae la cabecera de la fila requerida (`rowHeaders[row]`) y utiliza un bucle while para avanzar secuencialmente nodo por nodo a través del puntero `nextInRow` hasta encontrar la columna solicitada o pasarse de ella (`actual->col <= col`). En el peor de los casos, debe recorrer todas las celdas ocupadas de esa fila.
- **insertarCelda:**
 - **Complejidad:** $O(Kr + Kc)$
 - **Justificación:** La inserción requiere enlazar el nuevo nodo tanto horizontal como verticalmente. Para mantener el orden de la lista, el código llama a métodos auxiliares (`getPrevInRow` y `getPrevInCol`) que recorren la fila y la columna desde el inicio para encontrar exactamente qué nodo debe apuntar a la nueva celda y a qué nodo apuntará la nueva celda. Esto implica recorrer las listas correspondientes, sumando los tiempos de búsqueda en ambas dimensiones.
- **eliminarCelda:**
 - **Complejidad:** $O(Kr + Kc)$
 - **Justificación:** Al igual que la inserción, el código necesita "desenganchar" el nodo de la estructura. Para ello, utiliza nuevamente `getPrevInRow` y `getPrevInCol` con el objetivo de encontrar el nodo anterior en la fila y en la columna, puenteando así el puntero `next` de los nodos anteriores hacia los nodos siguientes antes de liberar la memoria con `delete`.

Recorrido del código

Función de inserción

```
void insertarCelda(int row, int col, const string& valor)
{
    expandirCabeceras(row, col);

    // if exists
    CellNode* actual = rowHeaders[row];
    while (actual != nullptr && actual->col <= col) {
        if (actual->col == col) {
            actual->value = valor;
            return;
        }
        actual = actual->nextInRow;
    }
}
```

```
CellNode* nuevoNodo = new CellNode(row, col, valor);
```

```
if (rowHeaders[row] == nullptr || rowHeaders[row]->col > col) {
    nuevoNodo->nextInRow = rowHeaders[row];
    rowHeaders[row] = nuevoNodo;
}
else {
    CellNode* prevRow = getPrevInRow(row, col);
    nuevoNodo->nextInRow = prevRow->nextInRow;
    prevRow->nextInRow = nuevoNodo;
}
```

```
if (colHeaders[col] == nullptr || colHeaders[col]->row > row) {
    nuevoNodo->nextInCol = colHeaders[col];
    colHeaders[col] = nuevoNodo;
}
else {
    CellNode* prevCol = getPrevInCol(row, col);
    nuevoNodo->nextInCol = prevCol->nextInCol;
    prevCol->nextInCol = nuevoNodo;
}
```

Primero nos aseguramos que los vectores rowHeaders y colHeaders tengan el tamaño suficiente para los índices solicitados. Si no existen, crea los espacios y los pone en nullptr.

Después se hace una búsqueda de celda existente, los punteros no se actualizan, solo se usa el puntero actual para recorrer la fila (actual = actual->nextInRow). Si actual->col == col, la celda ya existe. Se actualiza el valor y la función termina. No hay cambios en los enlaces.

Se crea el nodo en memoria. En este momento, sus punteros internos nextInRow y nextInCol están vacíos.

Si la fila está vacía (nullptr) o si la columna del nuevo nodo es menor que la del primer nodo actual (debe ir al principio para mantener el orden) entonces el siguiente del nuevo nodo será igual a la cabecera y la cabecera será igual al nuevo nodo.

Si la celda va en medio o al final. Primero buscamos el nodo previo. El siguiente del nuevo nodo apuntará a lo mismo que apuntaba el siguiente del previo. El siguiente del previo ahora apuntará al nuevo nodo.

Si la columna está vacía o si la fila del nuevo nodo es menor que la del primer nodo de esa columna entonces el siguiente (abajo) del nuevo nodo apuntará a donde apunta actualmente la cabecera de la columna (el nuevo nodo se coloca arriba de todos). La cabecera de la columna ahora apuntará al nuevo nodo.

Sino buscamos el nodo previo superior. El siguiente (abajo) del nuevo nodo apuntará a lo mismo que apuntaba el siguiente del previo superior. El siguiente del previo superior ahora apuntará al nuevo nodo.

Recorrido del código

Función de eliminación de celda

```
void eliminarCelda(int row, int col)
{
    if (row >= rowHeaders.size() || col >= colHeaders.size()) return;
    CellNode* nodoAEliminar = nullptr;

    if (rowHeaders[row] != nullptr && rowHeaders[row]->col == col) {
        nodoAEliminar = rowHeaders[row];
        rowHeaders[row] = nodoAEliminar->nextInRow;
    }
    else {
        CellNode* prevRow = getPrevInRow(row, col);
        if (prevRow != nullptr && prevRow->nextInRow != nullptr && prevRow->nextInRow->col == col) {
            nodoAEliminar = prevRow->nextInRow;
            prevRow->nextInRow = nodoAEliminar->nextInRow;
        }
    }

    if (colHeaders[col] != nullptr && colHeaders[col]->row == row) {
        colHeaders[col] = colHeaders[col]->nextInCol;
    }
    else {
        CellNode* prevCol = getPrevInCol(row, col);
        if (prevCol != nullptr && prevCol->nextInCol != nullptr) {
            prevCol->nextInCol = prevCol->nextInCol->nextInCol;
        }
    }

    if (nodoAEliminar == nullptr) return;

    delete nodoAEliminar;
}
```

Primero verificamos si la fila o columna existen. Si no existen, no hay nada que borrar después creamos una "etiqueta" temporal llamada nodoAEliminar para guardar la memoria que vamos a borrar al final.

Si la cabecera de la fila apunta justo a la celda que buscamos entonces la etiqueta nodoAEliminar ahora guarda la dirección de esa primera celda. La cabecera de la fila ahora apunta al siguiente que tenía esa celda. Si no buscamos al previo (el que está a la izquierda). Guardamos en nodoAEliminar la celda que está justo a la derecha del previo y el siguiente del previo ahora apunta al siguiente del nodo que vamos a borrar.

Si el nodo que queremos borrar es el que está hasta arriba de la columna entonces la cabecera de la columna ahora apunta al siguiente (el de abajo) del nodo actual. (El techo ahora sostiene directamente al que estaba debajo). Si no buscamos al previo superior (el que está justo arriba). El siguiente del previo superior ahora apunta al siguiente del siguiente. (El nodo de arriba ignora al que vamos a borrar y se conecta directamente con el que está más abajo).

Si después de todo no encontramos el nodo, no hacemos nada. Como ya nadie en las filas (horizontal) ni nadie en las columnas (vertical) apunta a nuestro nodo, ejecutamos delete. La memoria se libera y el nodo desaparece para siempre.

Recorrido del código

Función de eliminación de fila/columna

```
void eliminarFila(int row) {  
    if (row >= rowHeaders.size() || rowHeaders[row] == nullptr) return;  
    CellNode* actual = rowHeaders[row];  
    while (actual != nullptr) {  
        int col = actual->col;  
        CellNode* siguiente = actual->nextInRow;  
        eliminarCelda(row, col);  
        actual = siguiente;  
    }  
}
```

Primero si la fila no existe o si la cabecera de la fila no apunta a nadie (está vacía), no hay nada que borrar y nos salimos.

Creamos un puntero viajero llamado actual que empieza apuntando a donde apunta la cabecera que sera nuestro punto de partida.

Empezamos un ciclo que se repetirá mientras nuestro puntero viajero siga apuntando a un nodo real.

El puntero siguiente ahora apunta a quien el nodo actual tiene a su derecha.

Llamamos a la función eliminarCelda.

Ahora nuestro viajero pasa a apuntar al nodo que guardamos antes. Así saltamos al siguiente eslabón de la fila hasta que no quede ninguno.

```
void eliminarColumna(int col) {  
    if (col >= colHeaders.size() || colHeaders[col] == nullptr) return;  
    CellNode* actual = colHeaders[col];  
    while (actual != nullptr) {  
        int row = actual->row;  
        CellNode* siguiente = actual->nextInCol;  
        eliminarCelda(row, col);  
        actual = siguiente;  
    }  
}
```

Primero si la columna no existe o la cabecera de la columna no apunta a nadie (está vacía), terminamos.

El puntero viajero actual empieza apuntando a donde apunta la cabecera de la columna.

El ciclo continuará mientras actual siga apuntando a un nodo.

Guardamos el "hilo" de la columna. El puntero siguiente ahora apunta al nodo que está justo debajo del actual.

De nuevo, delegamos el trabajo pesado. Esta función rompe los enlaces y libera la memoria de la celda donde estamos parados.

El viajero ahora apunta al nodo de abajo, y el ciclo se repite.



Demostración en vivo de la aplicación





**Thank You
Very Much**

